

| Generating images

Lecture 7 – Autoencoders, VAEs, and a glance at diffusion

Dr Thomas Robinson

August 11, 2026

Warm-up · three questions from last time

≈ 5 MIN · SHOUT IT OUT

1. A 28×28 grayscale image goes through a 5×5 convolution, stride 1, no padding. What size is the output?
2. How many parameters does one 3×3 kernel on a grayscale input have? Count the bias.
3. Why does a convolutional layer need so many fewer parameters than a fully-connected layer on the same image?

Warm-up · answers

WORKED ANSWERS

1. **24×24.** With stride 1 and no padding, each side shrinks by the kernel width minus one: $28 - 5 + 1 = 24$.
2. **10.** Nine weights in the 3×3 grid, plus one bias: $9 + 1 = 10$.
3. **Weight sharing.** A kernel looks at one small window at a time, and the *same* weights slide across the whole image. A fully-connected layer gives every pixel its own weight; the kernel reuses its 10 numbers everywhere.

Breaking news!

'My AI agent booked a gym class — by booting out someone else'

- Australian man asks Claude agent to book him a gym class
- Classes fully booked but notices flaw allowing it to book unreleased slots
- Then prompted to bump man up existing waiting list

Copy-paste no more: Anthropic puts invisible watermarks on Claude text under EU rules

- Subtly alter the distribution of tokens/pixels in an invisible way
- Likely a defence against distillation
- I suspect this will be jail-broken fairly quickly

(I think you folks could code the latter up after today's session!)

| Part 1 · Generative vs predictive

Two flavours of modelling — quick reminder

Discriminative

Estimate $P(\mathbf{y} \mid \mathbf{X})$.

- Given features, predict the label.
- The CNN classifier from yesterday: $P(\text{class} \mid \text{image})$.
- Logistic regression, random forests, most “applied ML”.

GENERATIVE

Estimate $P(\mathbf{X})$ — and sample from it.

- Learn the *distribution of the data*.
- Sample from it to make new examples.
- DALL·E, Stable Diffusion, ChatGPT.

Generative modelling is harder – it’s a bigger ask. But it’s how we make new images, text, audio, molecules, and code.

Examples — across CS and social science

CS / industry

- DALL·E / Stable Diffusion — text-to-image.
- ChatGPT / Claude — text generation.
- MusicLM — audio.
- AlphaFold 2 — protein structures.
- Genie/Sora — video.

Social science

- **Synthetic survey data** – privacy-preserving release.
- **MIDAS** – missing-data imputation ([Lall & Robinson, 2022](#)).
- **Data augmentation** – generate extra training examples for small-N studies.
- **Counterfactual scenarios** – “what would this image look like under different lighting / demographics?”

Why we care

A model that can sample from $P(\mathbf{X})$ – the data distribution itself – is *also* a model that:

- Knows what's "in-distribution" vs unusual
- Can fill in missing parts of a partial observation
- Provides a principled basis for **simulation** in social science

Generative modelling is a *richer* object than discrimination. It is also harder to evaluate, which we'll return to in Part 3.

A first attempt — dimensionality reduction

BUILD UP TO GENERATION, SLOWLY, WITH A FAMILIAR TASK

Suppose we have 5-feature data and we want to compress to **2 dimensions**.

TRY

Sketch a neural network that takes a 5-feature input and produces a 2-feature output. What's the obvious architecture?

THE NATURAL SHAPE

A network with **decreasing widths** — $5 \rightarrow 3 \rightarrow 2$. The end becomes a “bottleneck”.

But there's a problem

- Where do we get the **target labels** for those 2 latent features?

We don't observe them and they don't exist in the dataset.

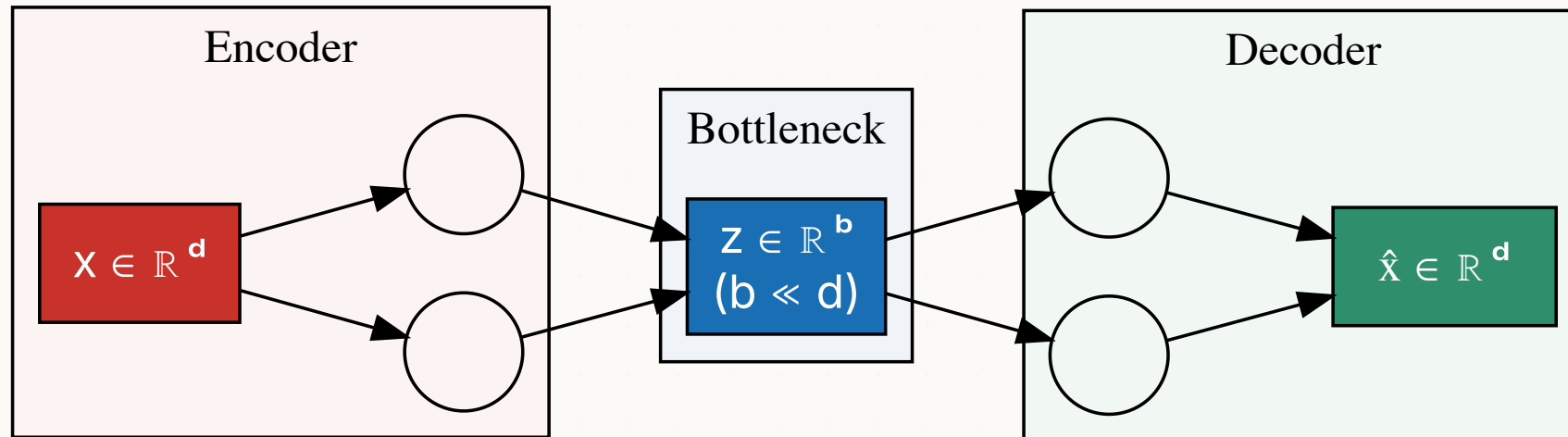
No labels = no loss = no backprop.

The autoencoder trick

Use the input as its own target.

Train the network to **reconstruct** its input *through* a narrow bottleneck.

The smaller number of features in the middle force the model to learn a compressed representation.



Reconstruction loss

Loss — how close is the reconstruction to the input?

$$\mathcal{L}_{\text{recon}}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2$$

Here d is the number of features.

Still, we train end-to-end with backprop, just like any other network.

At inference time:

- We can run the encoder to get the latent representation ($\mathbf{z}$)

Autoencoders are brittle

DEEP NETWORKS ARE LAZY!

TRY

What's the *easiest* function for an autoencoder to learn?

THE HONEST ANSWER

The identity function. Reconstruction loss = 0 when $\hat{\mathbf{x}} = \mathbf{x}$.

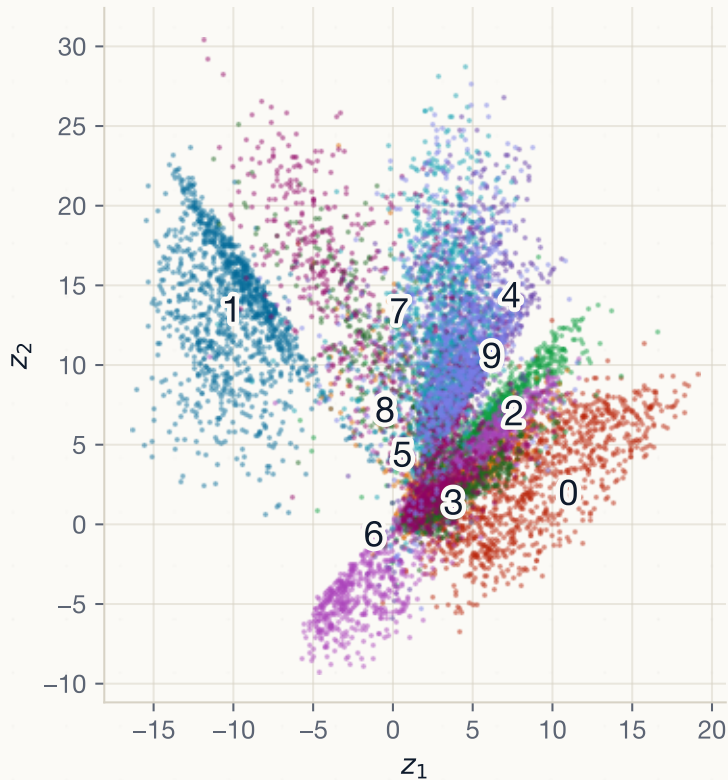
If the bottleneck isn't tight enough, the model finds shortcuts and the latent space becomes uninformative.

- *Deep nets are lazy!*

The latent space has **no structure we can sample from** so anything not in the training data is out-of-distribution.

Seeing the latent space

A REAL AE: 784 → 128 → 32 → 2 → 32 → 128 → 784, TRAINED ON MNIST



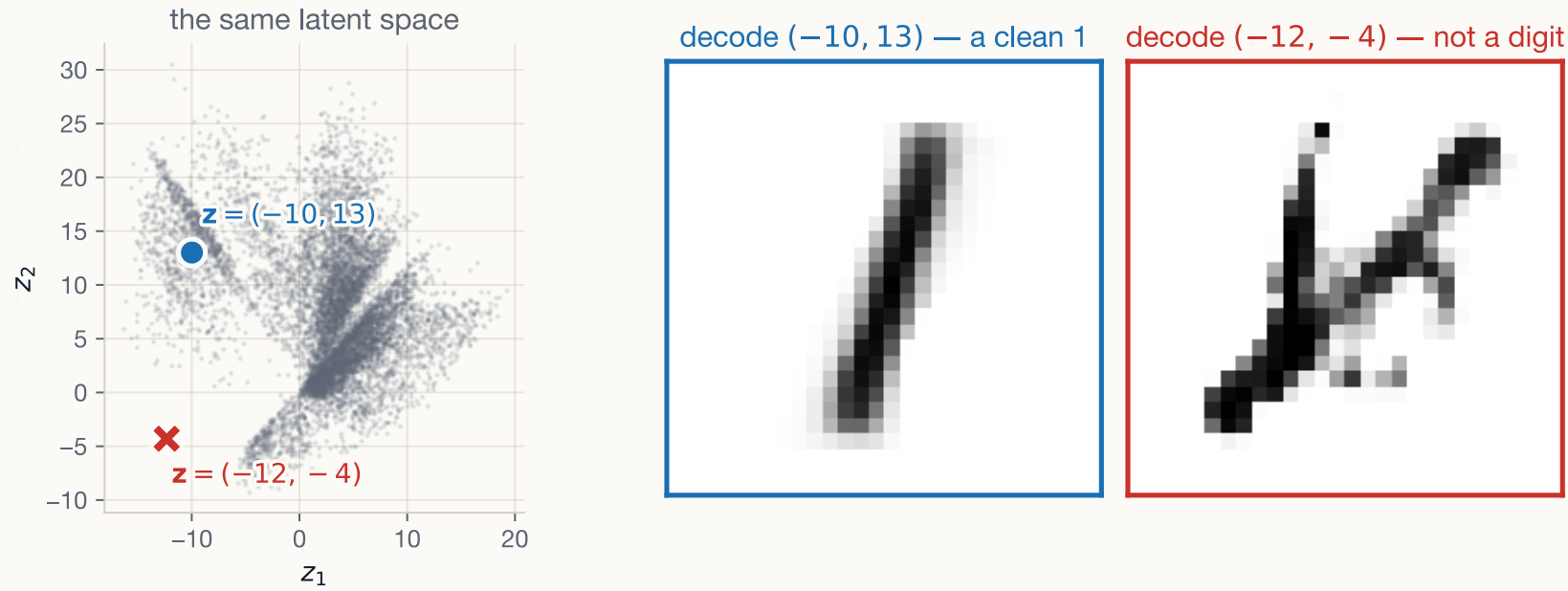
Train the autoencoder, then run the 10,000 held-out digits through the **encoder**. Each image is now a point (z_1, z_2) — coloured by its label:

- **Clusters** appear suggesting the model has found structure!
 - The labels were **never shown to the model**.
- The clusters are **uneven and elongated**; they overlap near the origin, with **gaps** elsewhere.
- The space *outside* the point cloud has never been seen by the decoder.

Ten epochs of Adam on the 60,000 training images; held-out reconstruction MSE 0.045.

Decoding the gaps

LET'S TRY SAMPLE FROM THE GAPS



The blue point sits in the middle of the **1 cluster** — the decoder turns it into a clean 1.

The red point sits in a **gap** and it returns garbage.

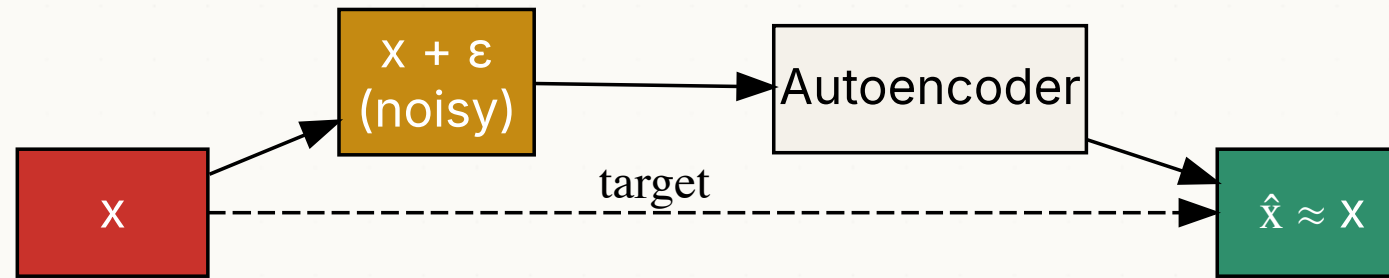


Note

An autoencoder's latent space is a **consequence** of training. It happens to compress — it doesn't necessarily mean it is sampleable.

Denoising autoencoders

A FIRST FIX



Corrupt the input. Ask the model to reconstruct the *clean* version.

- Forces the network to learn **structure**, not memorisation.
- Useful in its own right (denoising old photographs).
- A stepping stone to **VAEs** (next) and **diffusion** (later).

| Plenary · autoencoder forward pass

Plenary · autoencoder forward pass

≈ 8 MIN · IN PAIRS

A TINY AUTOENCODER

Input: $\mathbf{x} = \begin{bmatrix} 2 & 4 \end{bmatrix}$ (2 features).

Encoder: $\mathbf{W}_e = \begin{bmatrix} 0.5 & -0.5 \end{bmatrix}$, $b_e = 0$, identity activation.

So $\mathbf{z}$ is a **single number** — a 2D-to-1D bottleneck.

Decoder: $\mathbf{W}_d = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$, $\mathbf{b}_d = \begin{bmatrix} 0 & 1 \end{bmatrix}$, identity.

COMPUTE:

1. The latent value $\mathbf{z}$.
2. The reconstruction $\hat{\mathbf{x}}$.
3. The reconstruction loss $\mathcal{L} = \frac{1}{2} \sum (x_j - \hat{x}_j)^2$.

Five minutes.

Plenary · answers

ENCODER

$$\mathbf{z} = \mathbf{x}\mathbf{W}_e^\top + b_e = \begin{bmatrix} 2 & 4 \end{bmatrix} \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix} = (1) + (-2) = -1$$

DECODER

$$\hat{\mathbf{x}} = \mathbf{z}\mathbf{W}_d^\top + \mathbf{b}_d = -1 \begin{bmatrix} 1 & 0.5 \end{bmatrix} + \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 0.5 \end{bmatrix}$$

LOSS

$$\mathcal{L} = \frac{1}{2} [(2 - (-1))^2 + (4 - 0.5)^2] = \frac{1}{2} [9 + 12.25] = 10.625$$



Tip

A poorly-fit autoencoder. Backprop would update both $\mathbf{W}_e$ and $\mathbf{W}_d$ to lower this. The mechanics are the **same** as last week — just a different output node and loss function.

| **Part 2 · Variational autoencoders**

What we want — and why a plain AE doesn't give it

GOAL

A latent space such that:

- Every point in it decodes to something **plausible**.
- Nearby points decode to similar examples.
- We can **sample randomly** to generate new examples.

Plain autoencoder

- Latent space has whatever shape the encoder chooses.
- Most points outside the “training cloud” decode to garbage.
- No sampling distribution we can rely on.

From AE to VAE — three changes

WE ONLY NEED TO CHANGE THREE THINGS

CHANGE 1 — ENCODER OUTPUT

Instead of a single point $\mathbf{z}$, the encoder outputs **two vectors**: a mean μ and a standard deviation σ .

In practice the encoder emits $\log \sigma^2$ rather than σ — it keeps the variance positive and is numerically stabler.

CHANGE 2 — SAMPLE, DON'T PASS

Instead of sending μ to the decoder, sample a $\mathbf{z}$ from $\mathcal{N}(\mu, \sigma^2)$ and decode *that*.

- Every forward pass is now slightly randomised.

CHANGE 3 — REGULARISE THE LATENT

Add a second loss term that pulls the encoder's (μ, σ) output towards $\mathcal{N}(\mathbf{0}, \mathbf{I})$ – so that *any* draw from a standard Gaussian is a plausible latent code.

The VAE idea (Kingma & Welling, 2014)

Force the latent space to look like a standard Gaussian $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Now we can sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, pass it through the decoder, and get a new example.

How does that look at the network level?

1. The encoder outputs **two** vectors: $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ — the parameters of a Gaussian.
2. Sample $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ — the latent code.
3. Decode $\mathbf{z}$ to a reconstruction.
4. **Penalise** the encoder if its output distribution drifts far from $\mathcal{N}(\mathbf{0}, \mathbf{I})$.



Introducing stochastic network components

Every part of the network is still a deterministic function of its inputs *except* the sampling step.

What is KL divergence — informally

WE MEET IT NOW BECAUSE THE VAE LOSS ON THE NEXT SLIDE IS BUILT FROM IT.

KL divergence $\text{KL}(p \parallel q)$ measures *how different* two probability distributions are.

- $\text{KL}(p \parallel p) = 0$ (no difference).
- Large when p is concentrated where q is small.
- **Asymmetric:** $\text{KL}(p \parallel q) \neq \text{KL}(q \parallel p)$.

THE ONE CLOSED FORM WE NEED

For a 1-D Gaussian measured against the unit Gaussian $\mathcal{N}(0, 1)$, the KL has a tidy closed form:

$$\text{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1)) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$$

Plug in $\mu = 0.5$ and $\sigma = 0.3$:

$$\text{KL} = \frac{1}{2}(0.25 + 0.09 - 1 + 2.408) = \frac{1}{2}(1.748) \approx 0.87$$



Note

We won't *derive* this formula and you're not expected to know how to. The key *concept* is that KL-divergence measures the “distance between two distributions”.

KL divergence with real numbers

TWO COINS

For discrete distributions, KL is a weighted sum of log-ratios:

$$\text{KL}(P \parallel Q) = \sum_i p_i \ln \frac{p_i}{q_i}$$

Take a fair coin $P = [0.5, 0.5]$ and a biased coin $Q = [0.9, 0.1]$:

$$\text{KL}(P \parallel Q) = 0.5 \ln \frac{0.5}{0.9} + 0.5 \ln \frac{0.5}{0.1} = -0.294 + 0.805 \approx 0.511 \text{ nats}$$

Swap the arguments and the answer changes:

$$\text{KL}(Q \parallel P) = 0.9 \ln \frac{0.9}{0.5} + 0.1 \ln \frac{0.1}{0.5} = 0.529 - 0.161 \approx 0.368 \text{ nats}$$

Different in each direction. That asymmetry is why we say **divergence**, not distance. A “nat” is one natural-log unit of information — base e ’s answer to the bit.

DOUBLE CHECK IT STILL HOLDS

What is $\text{KL}(P \parallel P)$?

ANSWER

Zero. Every ratio inside the log is $\frac{0.5}{0.5} = 1$, and $\ln 1 = 0$.

What this buys the VAE

Since we're using a normal distribution, the closed form is what we'll use inside the VAE:

$$\text{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, 1)) = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$$

This is just four arithmetic operations on the encoder's own outputs:

DRIFTED MEAN

$$\mu = 1, \sigma = 1:$$

$$\frac{1}{2}(1 + 1 - 1 - \ln 1) = \frac{1}{2}(1) = 0.5$$

Right spread, wrong centre. Penalty: 0.5 nats.

ON TARGET

$$\mu = 0, \sigma = 1:$$

$$\frac{1}{2}(0 + 1 - 1 - 0) = 0$$

The encoder's output *is* the unit Gaussian. No penalty.

Combining two loss terms

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathcal{L}_{\text{recon}}(\mathbf{x}, \hat{\mathbf{x}})}_{\text{decoder does its job}} + \underbrace{\beta \cdot \text{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))}_{\text{latent stays close to a unit Gaussian}}$$

Reconstruction term

Same as a plain autoencoder. Wants the decoded image to match the input.

KL TERM

The **regulariser** — the KL divergence we just met. Wants the encoder's output distribution to look like a unit Gaussian.

Without it, the network ignores the latent variability — collapses back to a plain autoencoder.



Tip

The β weight (*hyperparameter*) trades off reconstruction quality vs latent space organisation.

Tuning β — fidelity vs structure

$\beta \rightarrow 0$

KL term vanishes

- The model is essentially a plain autoencoder
- Reconstructions are sharp
- The latent space is unstructured and not sampleable

β large

Reconstruction term is dwarfed

- The encoder is forced to output something close to $\mathcal{N}(\mathbf{0}, \mathbf{I})$ regardless of the input
- The latent space is very normal but reconstruction become **blurry**

THE TRADE-OFF

Higher β = smoother latent + blurrier samples.

The “ β -VAE” literature (Higgins et al., 2017) shows that pushing β up has a side-effect: latent dimensions tend to **disentangle** — one dimension controls rotation, another lighting, etc.

The reparameterisation trick

A PAIN IN THE BACKPROPAGATION

Problem

Sampling $\mathbf{z} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ is **non-differentiable** — you can't backprop through "randomly draw a sample".

- Without backprop, we can't train the encoder.

Fix

Rewrite the sample as:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- $\boldsymbol{\mu}, \boldsymbol{\sigma}$ are produced by the encoder — **differentiable**.
- $\boldsymbol{\epsilon}$ is randomness — but it's an **input**, not a parameter.
- Gradients flow through $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ as usual.

This trick makes the entire model trainable with ordinary backprop.

Almost every modern generative model uses some version of it.

A VAE forward pass

A TOY EXAMPLE WITH CONCRETE NUMBERS

SETUP

- Input: $\mathbf{x} = \begin{bmatrix} 1 & 2 \end{bmatrix}$
- Latent dim: 1
- Encoder produces $\mu = 0.5, \sigma = 0.3$
- Sample $\epsilon = 0.7$
- Decoder weights $\mathbf{W}_d = \begin{bmatrix} 1 & 0.5 \end{bmatrix}$, bias 0

STEP-BY-STEP

1. **Reparameterise:** $z = \mu + \sigma\epsilon = 0.5 + 0.3(0.7) = 0.71$.
2. **Decode:** $\hat{\mathbf{x}} = z\mathbf{W}_d = \begin{bmatrix} 0.71 & 0.355 \end{bmatrix}$.
3. **Recon loss:** $\frac{1}{2} \left[(1 - 0.71)^2 + (2 - 0.355)^2 \right] \approx 1.39$.
4. **KL term:** with $\mu = 0.5, \sigma = 0.3$, the closed form gives $\frac{1}{2}(0.25 + 0.09 - 1 + 2.408) \approx 0.87$.
5. **Total:** $\mathcal{L} \approx 2.26$.



Where the randomness sits

Step 1's ϵ is the only stochastic element. Once it's drawn, everything else is deterministic

Generating with a trained VAE

SAMPLING

1. Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
2. Pass through the **decoder** (ignore the encoder).
3. Get a new image.

INTERPOLATION

1. Encode two real images to $\mathbf{z}_1, \mathbf{z}_2$.
2. Walk the line $\mathbf{z}(t) = (1 - t)\mathbf{z}_1 + t\mathbf{z}_2, t \in [0, 1]$.
3. Decode at each step. Watch the image morph smoothly between the two.



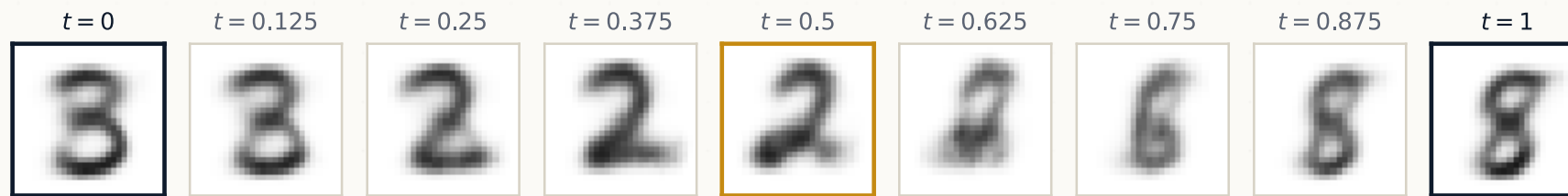
Tip

Smooth interpolation is the signature of a well-trained VAE. It tells you the latent space is *meaningful* — not just a clever lookup table.

Walking the latent space

INTERPOLATION WITH A REAL VAE — 2-D LATENT, TRAINED ON MNIST

Encode two real test images with a small trained VAE: a 3 becomes $\mathbf{z}_A = (1.1, -0.9)$, an 8 becomes $\mathbf{z}_B = (-1.0, -0.2)$. Decode nine points along the line $\mathbf{z}(t) = (1 - t)\mathbf{z}_A + t\mathbf{z}_B$:



Two things to notice:

- **Every stop is a plausible digit.** No garbage anywhere on the path — compare the plain autoencoder's gap point earlier. This is the KL term at work: the whole region around the origin decodes to digits.
- Notice halfway is a clean **2**, and around $t = 0.75$ we pass a **6**
 - The straight line from the 3s to the 8s crosses their patches of the plane

Both codes sit within about one unit of the origin — compared to AE's -15 to $+30$ span.

Latent arithmetic on complex features

THE CLASSIC FACES RESULT

Train a VAE on photos of faces. Then:

1. Average the codes of smiling faces: $\bar{\mathbf{z}}_{\text{smile}}$.
2. Average the codes of neutral faces: $\bar{\mathbf{z}}_{\text{neutral}}$.
3. Their difference is a **direction**: $\mathbf{v} = \bar{\mathbf{z}}_{\text{smile}} - \bar{\mathbf{z}}_{\text{neutral}}$.
4. Encode a new neutral face to $\mathbf{z}$, decode $\mathbf{z} + \mathbf{v}$ — the face smiles.

“Smiling” exists as a *line* in the latent space, which we can move along



Why the KL term makes this work

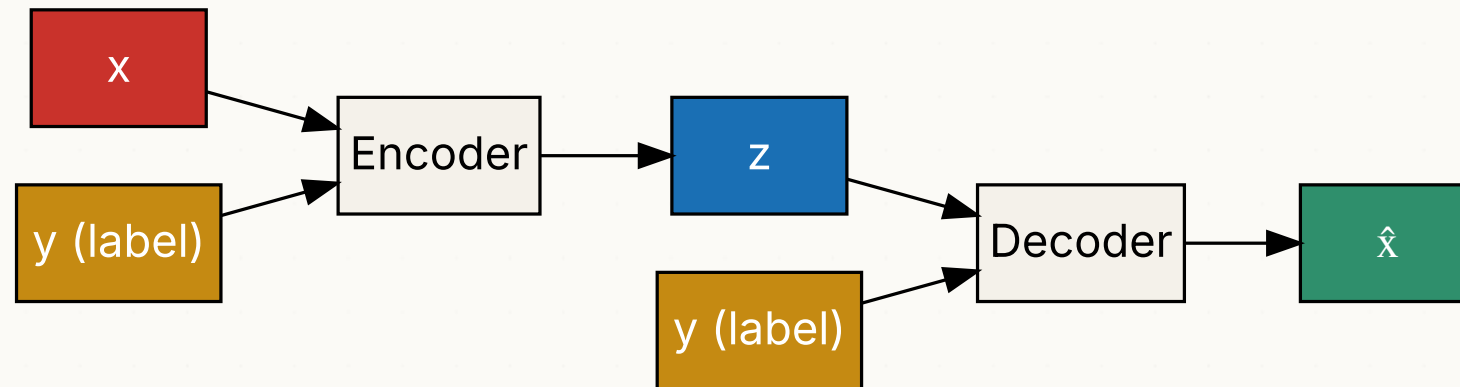
It packs the latents into one connected region around the origin, so lines between points stay on the data.

One thing a plain VAE cannot do

GENERATE A *CHOSEN* DIGIT

Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and decode — you get *a* digit. You cannot ask for a 7. You sample and take what comes.

The fix is the **conditional VAE** (CVAE): concatenate the label y to the encoder input *and* to the decoder input. The decoder now learns “reconstruct an image of THIS digit”.



Everything else — the sampling, the reparameterisation, the two losses — is unchanged.

Generating with a CVAE

AT GENERATION TIME

1. **Pick** the label — say $y = 7$.
2. **Sample** $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
3. **Decode** the pair $(\mathbf{z}, y)$.

The roles split: y carries *which* digit; $\mathbf{z}$ carries *how it is written* — slant, stroke width, loop size. Sample ten $\mathbf{z}$ values with $y = 7$ and you get ten different handwritten 7s.



Tip

If we replace the digit label with a (embedded) sentence of text, we can do text-to-image generation.

| Activity · VAE training step

Activity · VAE training step

≈ 8 MIN · IN PAIRS

THE PIECES OF A VAE FORWARD PASS

Order these steps **correctly** for a single training step on a single image $\mathbf{x}$:

- a. Compute reconstruction loss $\mathcal{L}_{\text{recon}}(\mathbf{x}, \hat{\mathbf{x}})$.
- b. Sample $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- c. Decode $\mathbf{z}$ to get $\hat{\mathbf{x}}$.
- d. Update parameters.
- e. Compute $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \epsilon$.
- f. Compute $\text{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I}))$.
- g. Run the encoder on $\mathbf{x}$ to get $\boldsymbol{\mu}, \boldsymbol{\sigma}$.
- h. `loss.backward()`.

Discuss in pairs. Then we'll line them up.

Activity · answer

THE CORRECT ORDER

1. **G** — encode $\mathbf{x} \rightarrow \mu, \sigma$.
2. **B** — sample ϵ .
3. **E** — reparameterise to $\mathbf{z}$.
4. **C** — decode $\mathbf{z} \rightarrow \hat{\mathbf{x}}$.
5. **A + F** — compute both losses; sum to total $\mathcal{L}$.
6. **H** — `loss.backward()` — fills gradients across encoder + decoder.
7. **D** — `optimizer.step()` — apply updates.



Tip

Notice the **forward pass** is now stochastic (it samples). The **backward pass** is still deterministic — gradients flow through μ, σ thanks to reparameterisation.

| Part 3 · Diffusion + social-science use

Why VAEs aren't the end of the story

VAEs work and they sample, but their outputs are often **blurry**.

- The reconstruction loss (MSE) averages over plausible decodings — and “average of plausible faces” looks “soft”.

By 2020 two other families had taken over the cutting edge:

- **GANs** (Goodfellow et al. 2014) — adversarial training. Sharp images but *really, really* hard to train.
- **Diffusion** (Ho et al. 2020) — denoising over many steps. Sharp and stable but slow.

Generative Adversarial Networks

The idea

Two networks play a game:

- The **generator** tries to make fake images that look real
- The **discriminator** tries to tell fakes from real
- They train *against* each other – one's gradient is the other's penalty

When the discriminator can't tell, the generator has learned the data distribution.

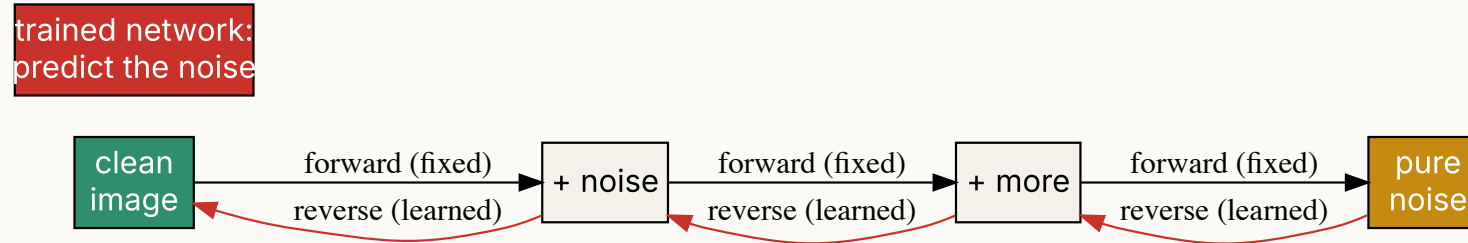


Why GANs lost steam

- **Mode collapse** — the generator finds one good fake and exploits it.
- **Training instability** — too-clever discriminator leads to useless gradients.
- **No likelihood** — you can't ask "how likely is this image?"

The 2014–2020 GAN era produced photorealistic images (StyleGAN, BigGAN, ...). But after 2020, diffusion was simpler and worked better.

Diffusion



- **Forward:** add a small amount of noise, repeatedly, until the image is pure noise.
- **Reverse:** train a network to predict the noise at each step and *subtract* it.
- **To generate:** start from random noise. Run the reverse process. Out comes an image.

Why diffusion works

- **Denoising is easier than generation.**
 - Predicting “a little less noise” is a tractable supervised problem.
- **The forward process is fixed**
 - Doesn't require training
- **Each reverse step is small**
 - The model can be huge without becoming unstable
- A generalisation of the **denoising autoencoder**



Denoising AE → diffusion model

Rather than “denoise in one shot”, we “denoise over T small steps”. The latter turns out to be more tractable.

Training diffusion — cheaper than it looks

THE FITTING ALGORITHM, UP CLOSE ([HO ET AL., 2020](#))

Two facts keep training affordable. **First, you never run the noising chain.** A sum of Gaussians is one bigger Gaussian, so there is a one-line shortcut to any step t :

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$\bar{\alpha}_t$ slides from ≈ 1 to ≈ 0 on a fixed schedule. Early on ($\bar{\alpha}_t = 0.99$): $\mathbf{x}_t = 0.995 \mathbf{x}_0 + 0.1 \boldsymbol{\epsilon}$, mostly image. Late ($\bar{\alpha}_t = 0.01$): $0.1 \mathbf{x}_0 + 0.995 \boldsymbol{\epsilon}$, mostly noise.

Second, one network handles every noise level — t goes in as an extra input. The whole loop:

1. Draw an image $\mathbf{x}_0$ from the data, a random step t , and noise $\boldsymbol{\epsilon}$.
2. Form $\mathbf{x}_t$ with the one-liner above.
3. Loss = $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$ — predict the noise, plain MSE.
4. Backprop, Adam step, repeat.

This is the Lecture 5 loop with a different target: no adversary to balance, no KL term in the loss you compute. One training step costs one forward + backward pass — the same as yesterday's CNN classifier.



Note

The cost is realised at sampling time. Generating one image runs the network $T \approx 1,000$ times *in sequence*. Hence we need fast samplers and/or diffusion chains run within autoencoder *latent spaces* (see Stable Diffusion).

Conditional generation — text-to-image

HOW DOES "A CAT WEARING A TOP HAT" BECOME AN IMAGE?

The diffusion model needs **two inputs** at each reverse step:

1. The current noisy image.
2. A **text embedding** of the prompt.

The model is trained to denoise *while staying true* to the text.

- Internally it uses **cross-attention** (Day 10) to inject the prompt into every denoising step.

Same architecture; one extra input.

- **Stable Diffusion** — UNet denoiser + CLIP text encoder.
- **DALL·E 3** — diffusion + OpenAI's text encoder.
- **Sora** — same idea, applied to video.

The text encoder is itself a (frozen) transformer. We come back to those on Day 10.

Classifier-free guidance ([Ho & Salimans, 2022](#))

Stable Diffusion and Midjourney both have a *guidance scale* setting.

TRAINING

Train **one** denoiser both with and without the caption: at random (say 10% of training steps), replace the caption with a blank. The model learns a conditional prediction *and* an unconditional one

- Notice the similarity to dropout

SAMPLING

At each reverse step, run the model twice and “push” the prediction *away* from the unconditional one:

$$\hat{\epsilon} = \epsilon_{\text{uncond}} + s (\epsilon_{\text{cond}} - \epsilon_{\text{uncond}})$$

- ϵ_{cond} — the predicted noise *with* the caption.
- ϵ_{uncond} — the predicted noise with the caption blanked.
- s — the **guidance scale**, chosen at sampling time.
- $\hat{\epsilon}$ — the prediction the sampler subtracts at this step.

The difference $\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}$ is the part of the prediction the caption is responsible for. Setting $s > 1$ amplifies that part.

Guidance scale in one dimension

Suppose at one denoising step the two predictions are $\epsilon_{\text{uncond}} = 0.2$ and $\epsilon_{\text{cond}} = 0.8$.

$s = 3$:

$$\hat{\epsilon} = 0.2 + 3 \times (0.8 - 0.2) = 2.0$$

The caption moved the prediction by 0.6; guidance moves it by 1.8 by amplifying its influence three-fold.

$s = 1$

$$\hat{\epsilon} = 0.2 + 1 \times 0.6 = 0.8 = \epsilon_{\text{cond}}$$

Plain conditional generation — no amplification.



Warning

The dial has a cost. Very high s trades variety + realism for prompt-adherence. Stable Diffusion uses $s = 7.5$ as its default.

Comparing the generative families

Family	Year	Quality	Speed	Likelihood?	Used for
VAE	2014	Blurry	Fast	Approximate	Compression, anomaly detection, imputation
GAN	2014	Sharp	Fast	No	Faces (StyleGAN), inpainting
Diffusion	2020	Sharpest	Slow	Approximate (variational bound); exact via probability-flow ODE	DALL·E, Stable Diffusion, Sora
Autoregressive	—	Excellent for text	Token-by-token	Yes	GPT, Claude — Day 10

The VAE first appeared as an arXiv preprint in 2013 and was published at ICLR 2014; the GAN in 2014; diffusion (DDPM) in 2020.

 **There is no single best strategy**

VAEs remain the right tool for many social-science applications (small data, need for likelihoods). Diffusion dominates image generation.

It depends on the **shape of your data** and **what you need from the model**.

Social-science application — MIDAS

Lall & Robinson ([2022](#)) — *The MIDAS Touch: Accurate and Scalable Missing-Data Imputation with Deep Learning*.

- The substantive problem: most real-world survey data has missing values. Common fixes (listwise deletion, MICE) are biased, slow, or both.
- MIDAS uses a **denoising autoencoder** trained on observed data.
- The network learns to “fill in” missing entries by recovering the structure of complete observations.

Empirically — more accurate than MICE on most datasets, dramatically faster on large ones, and handles time-series + mixed data types.



Tip

The pipeline you’ve now seen all the pieces for:

- Dropout
- Loss (reconstruction)
- Backprop
- Adam optimiser
- Denoising structure

Synthetic data — privacy, fairness, robustness

Privacy

- Researchers can't always share raw individual-level data
- A VAE trained on the data can produce **synthetic observations** that preserve statistics without exposing real ones
- Adoption is growing but there are questions about how well “private-by-design” holds in practice.

FAIRNESS

- Train a classifier on **augmented** data with rebalanced subgroups.
- Test for disparate impact
- Generate counterfactuals: “what would this image look like if the demographic feature changed?”



Synthetic data also enables misuse

Deepfakes, non-consensual imagery, fabricated evidence.

Evaluating generative models — open problem

Generative models are harder to evaluate than discriminative ones

- There's no simple "accuracy" number.

Common approaches:

- **Reconstruction loss** — measures fidelity but ignores diversity.
- **FID (Fréchet Inception Distance)** — distance between the *generated* and *real* distributions but needs real data to compare against.
- **Inception Score** — rewards images a pretrained classifier labels confidently (quality) while spreading across many classes (diversity), but no reference to real data.
- **Human evaluation** — gold standard, very expensive.
- **Downstream-task performance** — generate synthetic data, train a classifier on it, evaluate on real data.



The "right" metric depends on the use case.

A model that's good at reconstruction may be terrible at generating new samples. A model that's good at sampling may have a misaligned latent space.

Can synthetic people stand in for real ones?

A LIVE METHODS DEBATE ([ROBINSON & ZUBOK, 2026](#))

The tempting shortcut, “**silicon sampling**” ([Argyle et al., 2023](#)):

- Conditional generation using a demographic profile — *“a 52-year-old woman in Germany, secondary education, middle income...”* — and record the survey answers it gives back.
- Repeat 10,000 times: a “free” survey of any population

Before asking “is the model *good enough*?”, ask **where the information would come from**.

- A generator fit to **your own data** can only re-use what your sample already holds.
 - Analysing its output can never beat analysing the sample directly.
- An LLM **imports from outside** – whatever its training corpus says about people like the one in your prompt.
 - So the real question is: is that import *right for the population you’re targeting*?

The ecological fallacy in deep models

TRY — 60 SECONDS

Across countries, richer countries trust their police more. **Does it follow that, within a country, richer people trust the police more?**

ANSWER

No — that is the **ecological fallacy**: a between-group pattern need not hold within groups. In ~700,000 harmonised ESS/WVS responses, the within-country correlation of household income with trust in police is **0.03**. Across country *averages* it is **0.48**.

Ask an LLM for survey respondents country by country, and its respondents show a within-country correlation of **0.49** – the between-country value, stamped onto individuals.

The model builds each fake person *from the country profile*, so group-level patterns bleed into individual-level observations.

- Every one of the four LLMs we tested (four different developers) inflate the same way: 0.03 becomes 0.49–0.75.

When does synthetic data actually help?

A CHECKLIST FOR YOUR OWN PROJECTS

Judged against the thing you want to estimate, a generator can do one of three things:

- **Recover** information your incomplete data already identify
- **Re-express** a sample you already hold – automating a simulation you could have specified by hand. Convenient, but never informative
- **Distort** — by importing the wrong population, as silicon sampling does.

The output *looks* equally plausible in all three cases.

THREE RULES OF THUMB

1. LLMs are fine where only **plausibility** matters: pretesting questions, personas, piloting open-ended text.
2. For **estimation**, synthetic output enters only when *disciplined by real data*.
 - E.g. using just 200 real respondents, silicon sample estimates → correct coverage with a tighter interval than empirical data alone.
3. Before trusting any synthetic distribution, **compare its within-group correlations to real data**

Be careful!

SYNTHETIC MEDIA IS NO LONGER A RESEARCH CURIOSITY

The architecture you'll train in the lab is a (much smaller) version of what produces:

- **Non-consensual deepfake imagery** — Disproportionately affects women, including teenagers
- **AI-generated political ads**
- **Fabricated evidence**
- **Identity-laundering**

There are also enormous legitimate uses:

- Privacy-preserving health data
- Creative tools
- Scientific imaging

Take stock

WHERE YOU ARE AT THE END OF DAY 7

You can:

1. State the discriminative / generative distinction precisely.
2. Build a plain autoencoder for compression / denoising.
3. Explain why a plain AE is not a generative model.
4. Describe the VAE trick — Gaussian latent + KL regulariser + reparameterisation.
5. Explain the diffusion idea (denoising over steps).
6. Cite a concrete social-science generative model — MIDAS — and the evaluation problem.
7. Say when synthetic data helps — recover, re-express, or distort — and name the within-group check that separates the cases.

| **Lab brief · tomorrow morning**

Today's lab — 90 minutes

GOALS

Train a small VAE on MNIST and **generate new digits**.

You will:

1. Define a CNN encoder that outputs μ, σ .
2. Sample $\mathbf{z}$ using the reparameterisation trick.
3. Define a (transposed-convolution) decoder back to 28×28 .
4. Train with reconstruction + KL loss.
5. Sample from $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Plot the resulting digits.

STRETCH GOAL

Interpolate between two MNIST digits in latent space. Plot the morph as a grid of 10 images.

The smoothness of this morph is the most satisfying visual confirmation that the VAE has learned a meaningful latent space.

We leave images behind and turn to **text**. We'll ask how a network can turn a word into a vector — a **word embedding** — and build our first **character-level** model that generates text one letter at a time. It's the first of four lectures that build, step by step, towards a working GPT.



Tip

Optional reading tonight. Andrej Karpathy, [The Unreasonable Effectiveness of Recurrent Neural Networks](#) — a 2015 blog post that's still the best introduction to next-character generation.

- Argyle, L. P., Busby, E. C., Fulda, N., Gubler, J. R., Rytting, C., & Wingate, D. (2023). Out of one, many: Using language models to simulate human samples. *Political Analysis*, 31(3), 337–351.
- Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33, 6840–6851.
- Ho, J., & Salimans, T. (2022). Classifier-free diffusion guidance. *arXiv Preprint arXiv:2207.12598*.
- Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *International Conference on Learning Representations*.
- Lall, R., & Robinson, T. (2022). The MIDAS touch: Accurate and scalable missing-data imputation with deep learning. *Political Analysis*, 30(2), 179–196.
- Robinson, T. S., & Zubok, M. (2026). *Will synthetic data help?* [Unpublished manuscript].